CO4 – AT1 -CODING CHALLENGE

Course Title: Machine Learning Course
Code: ITA0609

Coure faculty :DR. Shri Vindhya A

Student Name: K.Vishwanathan

Reg No:192312063

•

1.Write a Python program to implement the K-Nearest Neighbour(KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

PROGRAM :

```
# K-Nearest Neighbour (KNN)
# Real-life example: Fruit Classification

import math

# Training data
X = [
    [150, 7],
    [160, 8],
    [140, 6],
    [180, 5],
    [170, 6],
    [190, 5]
]

# Class labels
Y = [0, 0, 0, 1, 1, 1]

# Test data
test = [155, 7]

# User enters K
k = int(input("Enter K value: "))

# Calculate Euclidean distance
distances = []

for i in range(len(X)):
    d = math.sqrt((X[i][0] - test[0]) ** 2 +
                  (X[i][1] - test[1]) ** 2)
    distances.append((d, Y[i]))

# Sort distances
distances.sort()

# Select K nearest neighbours
```

•

```
neighbors = distances[:k]

# Count classes
class_0 = 0
class_1 = 0

for d, label in neighbors:
    if label == 0:
        class_0 += 1
    else:
        class_1 += 1

# Prediction
if class_0 > class_1:
    prediction = 0
else:
    prediction = 1

print("Nearest Neighbors:", neighbors)
print("Predicted Class:", prediction)

if prediction == 0:
    print("Fruit: Apple")
else:
    print("Fruit: Orange")
```

OUTPUT :

Enter K value: 3

Nearest Neighbors: [(5.0, 0), (5.0990195135927845, 0), (15.033296378372908, 0)]
Predicted Class: 0
Fruit: Apple

==========
COMPANY PRODUCT DATASET
==========

Training Dataset:

Price ($)	Rating	Class
100	8.5	Good

•

120	**9.0**	**Good**
150	**8.0**	**Good**
200	**8.8**	**Good**
250	**9.2**	**Good**
500	**5.0**	**Poor**
550	**4.5**	**Poor**
600	**5.5**	**Poor**
650	**4.0**	**Poor**
700	**5.2**	**Poor**

2.Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

PROGRAM :

```
# Logistic Regression
# Real-life example: Email Spam Detection

import math

# Training data
# [Suspicious Words, Number of Links]
X = [
    [1, 1],
    [2, 1],
    [1, 2],
    [2, 2],
    [6, 5],
    [7, 6],
    [8, 5],
    [9, 7]
]

# Class labels
# 0 = Not Spam
# 1 = Spam
Y = [0, 0, 0, 0, 1, 1, 1, 1]
```

•

```
# Initial values
w1 = 0
w2 = 0
b = 0

learning_rate = 0.1
epochs = 1000

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + math.exp(-z))

# Gradient Descent
for epoch in range(epochs):

    dw1 = 0
    dw2 = 0
    db = 0

    for i in range(len(X)):

        z = w1 * X[i][0] + w2 * X[i][1] + b
        prediction = sigmoid(z)

        error = prediction - Y[i]

        dw1 += error * X[i][0]
        dw2 += error * X[i][1]
        db += error

    w1 = w1 - learning_rate * dw1 / len(X)
    w2 = w2 - learning_rate * dw2 / len(X)
    b = b - learning_rate * db / len(X)

# Test data
test_data = [
    [2, 1],
    [8, 6]
]

print("Final Weights:", round(w1, 2), round(w2, 2))
print("Bias:", round(b, 2))
```

```
print()

# Prediction
for test in test_data:

    z = w1 * test[0] + w2 * test[1] + b

    probability = sigmoid(z)

    if probability >= 0.5:
        result = "SPAM"
    else:
        result = "NOT SPAM"

    print("Suspicious Words:", test[0])
    print("Number of Links:", test[1])
    print("Sigmoid Output:", round(probability, 3))
    print("Classification:", result)
    print()

# Accuracy
correct = 0

for i in range(len(X)):

    z = w1 * X[i][0] + w2 * X[i][1] + b
    probability = sigmoid(z)

    if probability >= 0.5:
        prediction = 1
    else:
        prediction = 0

    if prediction == Y[i]:
        correct += 1

accuracy = (correct / len(X)) * 100

print("Accuracy:", accuracy, "%")
```

OUTPUT :

•

Final Weights: 1.36 0.02
Bias: -5.2

Suspicious Words: 2
Number of Links: 1
Sigmoid Output: 0.079
Classification: NOT SPAM

Suspicious Words: 8
Number of Links: 6
Sigmoid Output: 0.997
Classification: SPAM

Accuracy: 100.0 %